

Day 1: What I Actually Built

MRI k -Space Simulator — my own notes

I started Day 1 with an empty `mri/` folder and eight zero-byte files. By the end I had four working modules, three gates passed, and two bugs caught that would have wrecked me later if I had found them on Day 3 instead. These are my notes on what I built and why, so I do not forget the reasoning.

1 The engine: `kspace_core.py`

This is the foundation. Everything else in the project either multiplies something into what this file produces, or inverts something back out of it.

The whole idea is a sandwich. I go into k -space, I mess with it, I come back out. Same shifts on both sides:

```
image --> fft2 --> fftshift --> [ do all the damage here ] --> ifftshift --> ifft2 --> image
```

`to_kspace` does the first half, `from_kspace` does the second. That is it. Two functions, four lines of real work.

The `fftshift` part is the bit I had to think about. The raw FFT dumps the zero-frequency term in the *top-left corner* and wraps the negative frequencies around to the far side. There is nothing deep about that, it is just a storage convention. But every mask I write later assumes the middle of the matrix is the low-frequency region, so I shift it so the DC term sits in the centre and frequency grows outward. `ifftshift` undoes it on the way back.

Why I display $\log(1 + |K|)$ and never reconstruct from it

The DC term and its neighbours are astronomically bigger than everything else, often by a factor of a million, because most images are mostly smooth and smooth means low frequency. If I show $|K|$ raw I get one white pixel on a black square. So `log_magnitude` squeezes the range down for display. The `1+` keeps log away from zero. This is **display only**. It is not the real recipe any more, so it must never go back into a reconstruction.

I also front-loaded the annoying input cases instead of leaving them for Day 3: colour images get collapsed with the luminance formula, non-square images get zero-padded to square *before* resizing (resize first and the anatomy stretches), and a constant or all-black image returns zeros instead of dividing by zero in the normaliser. Four lines each now, versus hunting them down on my last afternoon.

How I know it works

The round-trip test is the one that matters. Image in, k -space, image out, no changes in the middle. If the transform is lossless the error should be essentially zero:

```
[round-trip]    max_error=5.551e-16
[parseval]     rel_error=1.180e-16
[dc]           K_centre=8064.7 pixel_sum=8064.7
```

5.55×10^{-16} is machine precision, so the transform loses nothing. Parseval says the energy matches in both domains. And the DC term coming out exactly equal to the sum of all my pixels is the sanity check that the centring worked, because that is literally what $K[0,0]$ means: set $u = v = 0$ in the formula, the exponential becomes 1, and the whole double sum collapses to “add up every pixel”.

2 The masks: `cartesian_sampling.py`

Undersampling sounds complicated and turns out to be one multiplication:

$$K_{\text{sub}}[u, v] = K[u, v] \cdot S[u, v]$$

where S is a matrix of ones and zeros the same size as k -space. One means I acquired that sample, zero means I skipped it. That is the entire trick. Everything after that is just choosing interesting patterns of ones and zeros.

I built four: **uniform** (keep every R -th row), **variable density** (fully sample the centre band, sparse at the edges), **random** (keep each row with probability $1/R$), and **corner cut** (keep a central disc, which is really a low-pass filter rather than row skipping).

I also wrote **achieved_acceleration** because the R I ask for is not the R I get. Variable density adds a fully sampled centre band on top of the skipping, so at $R = 4$ I actually measure $R_{\text{actual}} = 2.75$. Random is a coin flip, so it lands near R but not on it. Better to report the truth than pretend the number I typed is the number I got.

Bug 1: I was throwing away the DC line

`mask[:,R]` counts rows from zero. But after `fftshift` the DC term sits at row `rows//2`, which is 128. So row 128 only survives when $128 \bmod R = 0$. For $R = 3, 5, 6, 7$ it does not, and I was silently deleting the single brightest, most important sample in all of k -space. The damage was worse than it sounds. At $R = 3$ I was keeping only 22.6% of the energy while $R = 4$ kept 50.1% — so *less* acceleration was giving me *far* worse data, which makes no sense. And the mean brightness dropped from 0.1231 to 0.0727, about 41% too dark, because the DC term *is* the average brightness.

The fix is one line: anchor the pattern on the DC row instead of row 0.

```
mask[(rows//2) % R :: R, :] = 1
```

Now every R keeps the centre line, energy falls monotonically as it should ($66\% \rightarrow 55\% \rightarrow 50\% \rightarrow 48\%$), and brightness holds steady at 0.1231.

It slipped through because the self-test only checked $R = 2$, which happens to be one of the values where the bug does not bite. I added a loop over $R = 1..8$ so it cannot come back.

3 Predicting artifacts: `psf_analyzer.py`

This is the part that made the whole thing click for me. The **point spread function** is what you get when you inverse-transform the mask *on its own*, with no image involved at all. It tells me what artifact that mask will produce before I reconstruct anything.

For a uniform mask at acceleration R , the PSF is exactly R spikes spaced N/R rows apart. My tests confirm it:

```
[R=1] ghosts=1 offsets=[0] sidelobe=0.00e+00
[R=2] ghosts=2 spacing=128 predicted=128 sidelobe=1.000
[R=4] ghosts=4 spacing=64 predicted=64 sidelobe=1.000
[R=8] ghosts=8 spacing=32 predicted=32 sidelobe=1.000
```

Those spikes *are* the ghosts. Convolving my image with R spikes is what stamps R shifted copies of the anatomy on top of each other. So the ghosting is not some mysterious artifact, it is just what the mask looks like in image space.

I wrote `peak_sidelobe` to put a number on it: the height of the strongest ghost relative to the main lobe. This is where it gets interesting:

Mask at $R = 4$	Sidelobe	What that means
uniform	1.000	a ghost exactly as bright as the real anatomy
variable density	0.591	weaker, because the centre is fully sampled
random	0.313	no single strong ghost, energy is scattered

The uniform sidelobe being exactly 1.000 is the whole problem with regular skipping: the ghost is not faint, it is a full-strength copy. Randomising the rows drops it to 0.313 — the same information is missing either way, but now it shows up as noise-like grain instead of a clean duplicate. That is the entire foundation of compressed sensing MRI, and I can see it in my own numbers.

I also test the direction, because getting it backwards is the classic mistake. Skipping *rows* must ghost *vertically*. My check measures energy away from the main lobe: the central column has 0.500 and the central row has exactly 0. If those ever swap, my mask is transposed.

4 Scoring it: `metrics.py`

Three numbers so “this looks worse” becomes something I can plot against a slider. MSE (0 is perfect), PSNR in dB (higher is better, and *infinite* for an exact match, which the GUI has to be ready to print), and SSIM (compares local structure, tracks what the eye sees better than MSE does). Plus an error map, which is just the absolute difference per pixel, for the heatmap panel.

The test that actually matters here is monotonicity — quality must fall as R rises, every time:

```
[R=1] mse=0.00000 psnr=321.75 ssim=1.0000
[R=2] mse=0.01990 psnr= 17.01 ssim=0.7103
[R=4] mse=0.02934 psnr= 15.33 ssim=0.5115
[R=8] mse=0.03328 psnr= 14.78 ssim=0.4496
```

$R = 1$ giving 321 dB instead of infinity is just the floating-point round-trip error showing up. That is expected and fine.

5 The GUI spike, and why I did it on Day 1

The plan had me spend 90 minutes on a throwaway GUI on Day 1 evening, which felt like a detour at the time. One window, one slider, one matplotlib canvas. Not the real app, which is Day 3 work. The only point was to prove PySide6 opens, that a matplotlib canvas embeds in a Qt layout, and that a slider signal reaches my engine.

It paid for itself immediately.

Bug 2: the app would not start at all

```
AttributeError: '_SixMetaPathImporter' object has no attribute '_path'
```

PySide6 installs an import hook that inspects every module imported after it. Matplotlib’s Qt backend pulls in `dateutil`, which pulls in `six`, and `six.moves` modules are synthetic, so the hook chokes trying to read their source. Crash on startup, before a single window

appears.

Import order alone did not fix it — I tried. What works is fully loading the date chain before Qt gets its hook in:

```
import matplotlib
matplotlib.use("QtAgg")
import matplotlib.dates # <-- this line, before the Qt backend
from matplotlib.backends.backend_qttagg import FigureCanvasQTAgg
```

One line. But finding it on Day 3 morning, with the whole interface still to build, would have cost me hours I did not have.

That is the actual argument for the spike. It is not about the code, which I delete. It is about moving the unknown from “six hours left” to “two days left”.

6 Where I stand

Gate	What it proves	Result
1	round-trip, Parseval, DC, edge cases	pass, 5.55×10^{-16}
2	PSF spikes, spacing, ghost direction	pass, exact at $R = 1, 2, 4, 8$
3	window opens, canvas embeds, slider wired	pass

Four modules done: `kspace_core`, `cartesian_sampling`, `psf_analyzer`, `metrics`. Still empty: `filters`, `trajectories`, `gridding`, `pipeline`, and `app.py`.

Day 2 is `filters` and noise in the morning, then `trajectories`, then `gridding` in the afternoon — which is the one that can eat a whole day, so I use `scipy.griddata` and do not hand-roll a kernel. Evening is `pipeline.py`, which is the single function the GUI will call. Once that runs headless across every mask, window and trajectory combination, the project is functionally complete and Day 3 is just presentation.

Run any module with `python -m mri.<name>`. Not the file path — these import from each other, and running a file directly puts `mri/` on the path instead of the project root, so the imports break.